

Product Requirements Document (PRD)

Motor Inteligente de Segmentación de Clientes

Alan Alejandro Betancourt Ramírez

10 de septiembre de 2026

Contents

1. Descripción general del producto	4
2. Problema a resolver	4
3. Objetivo general	4
4. Objetivos específicos	4
5. Principios de arquitectura	5
6. Usuarios del sistema	5
6.1 Marketing	5
6.2 Ventas	5
6.3 Analistas	6
6.4 Administradores	6
6.5 Directivos	6
7. Fuentes de información soportadas conceptualmente	6
8. Arquitectura de integración y adaptadores	6
9. Contrato interno de datos	7
9.1 Customer	7
9.2 Transaction	7
9.3 Interaction	7
9.4 CampaignEvent	7
10. Información de clientes	8
11. Procesamiento y calidad de datos	8
12. Ingeniería de características	8
13. Análisis RFM	9
14. Motor de Machine Learning	9

15. Interpretación de segmentos	9
16. Audiencias dinámicas	10
17. Recomendaciones mediante Inteligencia Artificial	10
18. Backend	10
19. API del producto	11
20. Persistencia con PostgreSQL	11
21. Azure Machine Learning	11
22. Frontend	12
23. Dashboard y visualización	12
24. Azure AI Search	12
25. Autenticación, autorización y seguridad	12
26. Privacidad de datos	13
27. Docker, Git y CI/CD	13
28. Requerimientos funcionales	13
29. Requerimientos no funcionales	14
30. Validación del modelo	14
31. Métricas de producto	15
32. Arquitectura tecnológica propuesta	15
33. Integración futura con CRM reales	16
34. Riesgos	16
34.1 Calidad insuficiente de datos	16
34.2 Contrato interno mal definido	16
34.3 Segmentos poco interpretables	16
34.4 Integraciones externas	16
34.5 Dependencia de servicios cloud	16
34.6 Seguridad y privacidad	16
35. Estrategias de mitigación	16
36. Criterios generales de aceptación del producto	17
37. Evolución futura	17
38. Supuestos y decisiones confirmadas	18

Empresa: PluriOne S.A. de C.V.

Nombre comercial: Develop Talent & Technology

Proyecto: Desarrollo de un Motor Inteligente de Segmentación de Clientes

Versión: 1.1

Estado: Documento base de producto

1. Descripción general del producto

El Motor Inteligente de Segmentación de Clientes será una plataforma orientada al análisis automático de información comercial, transaccional y de comportamiento de clientes.

El producto deberá recibir información proveniente de distintas fuentes empresariales, transformarla a una estructura interna estable, preparar las variables necesarias para análisis y ejecutar modelos de Machine Learning capaces de identificar grupos de clientes con características similares.

Sobre los segmentos obtenidos, el sistema podrá generar descripciones y recomendaciones comerciales mediante Inteligencia Artificial Generativa, además de proporcionar interfaces para consulta, análisis y creación de audiencias.

La arquitectura deberá ser modular y desacoplada. **El núcleo del sistema no dependerá de un CRM específico ni de la estructura de una fuente concreta.** Toda fuente externa deberá integrarse mediante una capa de adaptación hacia un contrato interno de datos.

2. Problema a resolver

Las organizaciones pueden disponer de información de clientes distribuida entre CRM, ERP, plataformas de ventas, campañas, sitios web, bases de datos y archivos. Sin una capa de integración y análisis, esa información puede resultar difícil de aprovechar para identificar patrones de comportamiento y tomar decisiones comerciales.

La segmentación manual o basada únicamente en reglas estáticas puede provocar:

- campañas dirigidas a audiencias poco relevantes;
- baja personalización de estrategias comerciales;
- dificultad para identificar clientes de alto valor;
- dificultad para detectar clientes inactivos o en riesgo;
- desaprovechamiento del historial de compras e interacciones;
- dependencia de procesos manuales;
- dificultad para reutilizar el análisis cuando cambia la fuente de datos.

El producto busca resolver estos problemas mediante una arquitectura de integración desacoplada y un motor de segmentación basado en datos.

3. Objetivo general

Desarrollar un Motor Inteligente de Segmentación de Clientes capaz de integrar, normalizar y analizar información de clientes para identificar automáticamente grupos con características similares y generar información útil para marketing, ventas y análisis comercial.

4. Objetivos específicos

- Integrar información proveniente de diferentes fuentes mediante adaptadores.

- Definir un contrato interno de datos independiente de los sistemas externos.
- Preparar, validar y transformar información de clientes.
- Implementar ingeniería de características para segmentación.
- Implementar modelos de Machine Learning para agrupamiento de clientes.
- Evaluar técnica y comercialmente los segmentos obtenidos.
- Gestionar y desplegar modelos mediante Azure Machine Learning.
- Generar recomendaciones comerciales mediante Azure OpenAI Service.
- Exponer funcionalidades mediante un backend desarrollado con FastAPI.
- Almacenar información y resultados mediante PostgreSQL.
- Proporcionar una interfaz web mediante React.
- Permitir la creación y actualización de audiencias dinámicas.
- Mantener trazabilidad de las ejecuciones de segmentación.
- Facilitar la integración futura de CRM reales sin modificar el núcleo del motor.

5. Principios de arquitectura

El producto deberá respetar los siguientes principios:

1. **Desacoplamiento de fuentes:** el motor no debe consumir directamente el formato nativo de un CRM, ERP o archivo externo.
2. **Contrato interno estable:** toda fuente deberá convertirse a un modelo interno documentado.
3. **Adaptadores por fuente:** cada integración deberá implementar su propia traducción hacia el contrato interno.
4. **Separación de responsabilidades:** integración, persistencia, procesamiento, Machine Learning, API y frontend deberán mantenerse separados.
5. **Seguridad por backend:** secretos, credenciales y llamadas a servicios protegidos deberán permanecer fuera del frontend.
6. **Machine Learning antes que IA generativa:** la segmentación será responsabilidad del modelo de ML; Azure OpenAI se utilizará como complemento interpretativo.
7. **Validación local antes de despliegue:** los modelos deberán validarse antes de ser administrados o servidos mediante Azure Machine Learning.

6. Usuarios del sistema

6.1 Marketing

Podrá consultar segmentos y audiencias para diseñar campañas más específicas. Podrá analizar comportamiento, frecuencia de compra, valor comercial, preferencias e interacción con campañas.

6.2 Ventas

Podrá identificar grupos prioritarios, oportunidades comerciales y clientes con determinados patrones de valor o actividad.

6.3 Analistas

Podrán revisar resultados de segmentación, métricas, características de clusters y ejecuciones del modelo.

6.4 Administradores

Podrán gestionar usuarios, configuraciones, fuentes, permisos y aspectos operativos del producto.

6.5 Directivos

Podrán consultar indicadores generales, distribución de segmentos y métricas comerciales relevantes.

7. Fuentes de información soportadas conceptualmente

El producto deberá poder incorporar, mediante adaptadores, fuentes como:

- CRM;
- ERP;
- bases de datos empresariales;
- APIs REST;
- sistemas de ventas;
- plataformas de comercio electrónico;
- herramientas de marketing;
- herramientas de analítica;
- archivos CSV;
- archivos JSON;
- hojas de cálculo.

La incorporación de una nueva fuente no deberá obligar a modificar el motor de Machine Learning si el contrato interno permanece estable.

8. Arquitectura de integración y adaptadores

La capa de integración será responsable de obtener datos de sistemas externos y transformarlos al modelo interno del producto.

Cada fuente deberá contar con un adaptador responsable de:

- autenticarse cuando aplique;
- consultar o recibir datos externos;
- mapear nombres de campos;
- convertir tipos de datos;
- normalizar valores;
- validar campos obligatorios;
- reportar registros inválidos;
- entregar objetos compatibles con el contrato interno.

Ejemplo conceptual:

```
HubSpot -----> HubSpotAdapter -----+
Salesforce -----> SalesforceAdapter ----+
CSV -----> CsvAdapter -----+----> Contrato interno --> Núcleo del sistema
Base de datos ---> DatabaseAdapter -----+
```

El adaptador no deberá contener lógica de segmentación.

9. Contrato interno de datos

El sistema utilizará un modelo interno como frontera entre las fuentes externas y el núcleo de procesamiento.

El contrato podrá evolucionar mediante versiones documentadas, evitando cambios incompatibles sin control.

9.1 Customer

Campos conceptuales:

- customer_id
- age
- city
- registered_at

9.2 Transaction

Campos conceptuales:

- transaction_id
- customer_id
- purchased_at
- amount
- category

9.3 Interaction

Campos conceptuales:

- interaction_id
- customer_id
- interaction_type
- occurred_at

9.4 CampaignEvent

Campos conceptuales:

- campaign_id
- customer_id

- event_type
- occurred_at

Los campos opcionales podrán ser nulos. Los campos obligatorios deberán documentarse por versión del contrato.

10. Información de clientes

Dependiendo de la fuente disponible, el producto podrá utilizar información como:

General: ID, fecha de registro, edad, ubicación y variables demográficas disponibles.

Transaccional: fechas de compra, monto, producto, categoría, frecuencia y ticket promedio.

Preferencias: categorías, productos o marcas con mayor interacción o consumo.

Comportamiento: visitas, clics, búsquedas, sesiones, productos consultados y carritos abandonados.

Marketing: campañas recibidas, aperturas, clics, promociones y conversiones.

11. Procesamiento y calidad de datos

Antes de utilizar la información en el modelo, el sistema deberá permitir:

- detectar duplicados;
- tratar valores faltantes;
- validar tipos y formatos;
- transformar fechas;
- normalizar categorías;
- descartar o aislar registros inválidos;
- generar variables derivadas;
- escalar variables cuando el algoritmo lo requiera;
- registrar incidencias de calidad de datos.

El procesamiento deberá mantenerse separado de los adaptadores.

12. Ingeniería de características

El sistema deberá generar una vista de features por cliente a partir del contrato interno.

Como mínimo podrá incluir:

- días desde la última compra (recency_days);
- número de compras (frequency);
- gasto acumulado (monetary);
- ticket promedio;
- categoría dominante;
- tasa de respuesta a campañas;
- aperturas de correo;
- visitas web;
- carritos abandonados;

- métricas derivadas de engagement.

Las features deberán documentar su definición, fuente y tratamiento.

13. Análisis RFM

RFM se utilizará como base de análisis comercial y como conjunto mínimo de features:

- **Recency:** tiempo desde la última compra.
- **Frequency:** cantidad o frecuencia de compras.
- **Monetary:** valor económico acumulado.

El resultado podrá utilizarse directamente para análisis o como entrada a modelos de clustering.

14. Motor de Machine Learning

El producto contará con un componente independiente para segmentación de clientes.

Deberá permitir:

- recibir features preparadas;
- seleccionar variables;
- entrenar modelos;
- ejecutar modelos entrenados;
- generar clusters;
- asignar clientes a clusters;
- evaluar resultados;
- guardar metadatos de ejecuciones;
- manejar versiones del modelo.

K-Means será un algoritmo inicial de referencia, sin impedir el uso futuro de otros métodos de clustering cuando los datos lo justifiquen.

15. Interpretación de segmentos

Los clusters deberán interpretarse después de analizar sus características reales.

Ejemplos conceptuales de etiquetas comerciales:

- clientes frecuentes de alto valor;
- clientes nuevos;
- clientes ocasionales;
- clientes con baja actividad;
- clientes sensibles a promociones;
- clientes con alta interacción;
- clientes con riesgo de inactividad.

Las etiquetas no deberán asignarse antes de revisar los resultados reales del modelo.

16. Audiencias dinámicas

El sistema deberá permitir definir audiencias mediante condiciones sobre clientes, features o segmentos.

Ejemplos:

- clientes de determinada región;
- clientes con más de cierta cantidad de compras;
- clientes sin compra durante cierto periodo;
- clientes de una categoría específica;
- clientes pertenecientes a un segmento determinado.

Las audiencias deberán poder recalcularse cuando cambien los datos.

17. Recomendaciones mediante Inteligencia Artificial

Azure OpenAI Service podrá utilizarse para generar interpretaciones y recomendaciones comerciales a partir de estadísticas agregadas de los segmentos.

Podrá generar:

- descripción del segmento;
- perfil comercial;
- estrategias de fidelización;
- recomendaciones de retención;
- promociones sugeridas;
- ideas de cross-selling o up-selling;
- acciones de recuperación de clientes.

Azure OpenAI **no será responsable de decidir la pertenencia de clientes a clusters.**

18. Backend

El backend será desarrollado principalmente mediante Python y FastAPI.

Responsabilidades:

- ejecutar integraciones y adaptadores;
- validar solicitudes y datos;
- aplicar lógica de negocio;
- comunicarse con PostgreSQL;
- coordinar procesos de segmentación;
- consumir Azure Machine Learning;
- consumir Azure OpenAI;
- aplicar autenticación y autorización;
- exponer la API de la aplicación;
- registrar errores y operaciones relevantes.

19. API del producto

La API principal deberá proporcionar operaciones para:

- sincronizar o importar datos;
- consultar clientes;
- consultar segmentos;
- ejecutar segmentaciones;
- consultar ejecuciones;
- consultar recomendaciones;
- crear y consultar audiencias;
- consultar métricas.

Las rutas definitivas deberán documentarse mediante OpenAPI/Swagger generado por FastAPI y documentación adicional cuando sea necesario.

20. Persistencia con PostgreSQL

PostgreSQL será el sistema de almacenamiento principal.

Entidades conceptuales:

- customers;
- transactions;
- interactions;
- campaign_events;
- ingestion_runs;
- segmentation_runs;
- segments;
- customer_segments;
- audiences;
- recommendations;
- users;
- model_metadata.

Los datos externos no deberán confundirse con el modelo interno persistente. La información deberá pasar por adaptación y validación antes de considerarse parte del dominio interno.

21. Azure Machine Learning

Azure Machine Learning será utilizado para gestionar el ciclo de vida del modelo.

Podrá utilizarse para:

- entrenamiento y experimentación;
- registro de modelos;
- versionado;
- definición de ambientes y dependencias;
- despliegue;
- exposición mediante endpoints;

- administración de recursos de cómputo;
- monitoreo del modelo.

Los modelos podrán desarrollarse inicialmente en Python y registrarse en Azure ML una vez validados.

22. Frontend

La interfaz será desarrollada con React, preferentemente utilizando TypeScript/TSX.

El frontend deberá permitir:

- consultar clientes;
- visualizar segmentos;
- consultar detalle de segmento;
- mostrar recomendaciones;
- crear o consultar audiencias;
- visualizar métricas y gráficas.

El frontend únicamente deberá consumir la API del backend. No deberá contener credenciales de Azure ni llamar directamente a servicios protegidos.

23. Dashboard y visualización

El dashboard podrá mostrar:

- total de clientes;
- clientes por segmento;
- distribución de segmentos;
- gasto promedio;
- frecuencia promedio;
- recency promedio;
- principales categorías;
- métricas del modelo;
- evolución de ejecuciones;
- indicadores de campañas cuando existan datos.

Power BI podrá complementar el análisis ejecutivo cuando el alcance lo justifique, evitando duplicar innecesariamente visualizaciones ya disponibles en React.

24. Azure AI Search

Azure AI Search podrá incorporarse cuando exista una necesidad concreta de búsqueda semántica o recuperación avanzada de información. No deberá utilizarse únicamente por aparecer en el stack tecnológico.

25. Autenticación, autorización y seguridad

Microsoft Entra ID podrá utilizarse para autenticación empresarial.

El sistema deberá diferenciar autenticación de autorización y soportar roles como administrador, analista, marketing, ventas y visualizador cuando aplique.

Controles mínimos:

- secretos fuera del frontend;
- variables de entorno o gestores de secretos;
- conexiones seguras;
- validación de entradas;
- permisos según rol;
- protección contra acceso no autorizado;
- registro de operaciones relevantes;
- manejo controlado de errores.

26. Privacidad de datos

Cuando se utilicen datos reales, el sistema deberá considerar minimización, anonimización o pseudonimización, restricciones de acceso y políticas definidas por la organización.

Durante el desarrollo actual se utilizarán datos simulados.

27. Docker, Git y CI/CD

Docker podrá utilizarse para mantener ambientes consistentes entre desarrollo, pruebas y despliegue.

Git será obligatorio para control de versiones. GitHub podrá utilizarse como repositorio central.

GitHub Actions podrá automatizar pruebas, validaciones, construcción de imágenes y despliegues cuando el proceso esté suficientemente estable.

28. Requerimientos funcionales

RF-01. El sistema deberá recibir información estructurada de clientes desde fuentes externas.

RF-02. Cada fuente externa deberá integrarse mediante un adaptador o conector equivalente.

RF-03. Los adaptadores deberán convertir datos externos al contrato interno vigente.

RF-04. El sistema deberá validar los datos normalizados antes de incorporarlos al procesamiento.

RF-05. El sistema deberá registrar incidencias de datos inválidos sin detener innecesariamente el proceso completo.

RF-06. El sistema deberá almacenar datos normalizados en PostgreSQL.

RF-07. El sistema deberá limpiar y transformar información antes del análisis.

RF-08. El sistema deberá generar features para segmentación.

RF-09. El sistema deberá ejecutar modelos de clustering.

RF-10. El sistema deberá generar segmentos automáticamente.

- RF-11.** El sistema deberá asignar clientes a los segmentos resultantes.
- RF-12.** El sistema deberá almacenar resultados de segmentación y metadatos de ejecución.
- RF-13.** El sistema deberá permitir consultar segmentos y clientes asociados.
- RF-14.** El sistema deberá mostrar características principales de cada segmento.
- RF-15.** El sistema deberá permitir repetir una segmentación con datos actualizados.
- RF-16.** El sistema deberá manejar versiones del modelo.
- RF-17.** El sistema deberá generar recomendaciones por segmento mediante IA generativa.
- RF-18.** El sistema deberá permitir crear audiencias basadas en condiciones.
- RF-19.** El sistema deberá actualizar audiencias cuando cambien los datos.
- RF-20.** El sistema deberá proporcionar una API principal mediante FastAPI.
- RF-21.** El frontend deberá consumir la API principal.
- RF-22.** El sistema deberá permitir visualizar métricas generales.
- RF-23.** El sistema deberá administrar usuarios y permisos cuando el entorno lo requiera.
- RF-24.** El sistema deberá permitir incorporar nuevas fuentes creando nuevos adaptadores, sin modificar el núcleo del motor mientras el contrato interno permanezca compatible.
- RF-25.** El sistema deberá mantener trazabilidad de ingestiones y ejecuciones de segmentación.

29. Requerimientos no funcionales

- RNF-01. Seguridad:** proteger información, secretos y servicios contra accesos no autorizados.
- RNF-02. Privacidad:** aplicar controles de manejo de datos acordes al entorno de uso.
- RNF-03. Rendimiento:** mantener tiempos de respuesta adecuados para una aplicación web interactiva.
- RNF-04. Escalabilidad:** permitir incrementar el volumen de datos sin reconstruir completamente el sistema.
- RNF-05. Modularidad:** mantener separados integración, persistencia, procesamiento, ML, backend y frontend.
- RNF-06. Bajo acoplamiento:** ninguna fuente externa deberá dictar la estructura interna del motor.
- RNF-07. Adaptabilidad:** incorporar o sustituir fuentes mediante adaptadores.
- RNF-08. Mantenibilidad:** organizar código y módulos por responsabilidades claras.
- RNF-09. Trazabilidad:** identificar ingestiones, ejecuciones, modelos y resultados.
- RNF-10. Compatibilidad:** soportar navegadores modernos.
- RNF-11. Manejo de errores:** responder de forma controlada ante fallos externos o datos inválidos.
- RNF-12. Observabilidad:** permitir registros y monitoreo de componentes relevantes.
- RNF-13. Evolución del contrato:** versionar cambios incompatibles del modelo interno de datos.

30. Validación del modelo

Para clustering se podrán utilizar:

- Silhouette Score;
- Inertia;
- Davies-Bouldin Index;
- análisis de distribución;
- estabilidad entre ejecuciones;
- interpretación comercial de los clusters.

Un modelo no se considerará satisfactorio únicamente porque produce grupos. Los segmentos deberán presentar diferenciación suficiente y una interpretación razonable.

31. Métricas de producto

El producto podrá medir:

- registros recibidos;
- registros aceptados/rechazados;
- clientes procesados;
- tiempo de ingestión;
- tiempo de segmentación;
- número de segmentos;
- distribución de clientes;
- métricas de clustering;
- recomendaciones generadas;
- errores de integración;
- uso de funcionalidades.

32. Arquitectura tecnológica propuesta

Tecnologías principales:

- Python;
- FastAPI;
- React.js / TypeScript;
- PostgreSQL;
- Pandas;
- NumPy;
- Scikit-learn;
- Azure Machine Learning;
- Azure OpenAI Service.

Tecnologías complementarias según alcance:

- Azure AI Search;
- Power BI;
- Docker;
- GitHub;
- GitHub Actions;
- Microsoft Entra ID.

33. Integración futura con CRM reales

Para incorporar un CRM real se deberá:

1. identificar su mecanismo de acceso;
2. documentar su esquema o payload;
3. implementar un adaptador específico;
4. mapear sus campos al contrato interno;
5. validar la salida del adaptador;
6. integrar la fuente sin modificar el motor de segmentación mientras el contrato interno sea compatible.

La integración podrá realizarse mediante API REST, acceso autorizado a base de datos, exportaciones, webhooks u otros mecanismos proporcionados por el sistema externo.

34. Riesgos

34.1 Calidad insuficiente de datos

Datos incompletos o inconsistentes pueden afectar los resultados del modelo.

34.2 Contrato interno mal definido

Un contrato demasiado rígido o ambiguo puede dificultar integraciones futuras.

34.3 Segmentos poco interpretables

Un algoritmo puede producir grupos estadísticamente distintos pero comercialmente poco útiles.

34.4 Integraciones externas

APIs reales pueden presentar límites, permisos, cambios de versión o restricciones.

34.5 Dependencia de servicios cloud

Los servicios de Azure pueden introducir costos, límites o indisponibilidad.

34.6 Seguridad y privacidad

El uso futuro de datos reales requerirá controles más estrictos que los utilizados con datos simulados.

35. Estrategias de mitigación

- validar datos antes del procesamiento;
- documentar y versionar el contrato interno;
- mantener adaptadores separados del núcleo;

- probar el modelo localmente;
- evaluar distintas configuraciones de clustering;
- almacenar secretos de forma segura;
- registrar versiones de modelos;
- implementar pruebas de integración;
- documentar decisiones técnicas;
- priorizar componentes esenciales antes de tecnologías complementarias.

36. Criterios generales de aceptación del producto

El producto completo deberá poder:

- integrar al menos una fuente de datos mediante un adaptador;
- convertir la fuente al contrato interno;
- validar y persistir información normalizada;
- generar features de clientes;
- ejecutar un modelo de segmentación;
- producir grupos diferenciados;
- asignar clientes a segmentos;
- almacenar y consultar resultados;
- servir el modelo mediante infraestructura administrada;
- generar recomendaciones por segmento;
- visualizar resultados desde una interfaz;
- administrar audiencias;
- aplicar controles de acceso cuando corresponda;
- permitir incorporar nuevas fuentes sin rediseñar el núcleo.

37. Evolución futura

La arquitectura deberá permitir posteriormente:

- integración con CRM específicos;
- actualización automática o en tiempo real;
- integración bidireccional con CRM;
- predicción de abandono;
- Customer Lifetime Value;
- recomendación individual de productos;
- automatización de campañas;
- reentrenamiento automático;
- detección de anomalías;
- procesamiento de eventos en tiempo real;
- búsqueda semántica;
- personalización avanzada;
- integración con plataformas publicitarias;
- modelos adicionales de Machine Learning.

38. Supuestos y decisiones confirmadas

- No se contará con acceso directo a sistemas o datos reales de la empresa durante el desarrollo actual.
- Los datos de entrada serán simulados.
- Se simulará una fuente CRM mediante una API independiente del backend principal.
- La API simulada representará un sistema externo y podrá utilizar nombres o estructuras diferentes al contrato interno.
- El producto implementará un adaptador para convertir la fuente simulada al contrato interno.
- El motor de segmentación no dependerá de la estructura del CRM simulado.
- La integración futura con un CRM real se realizará mediante nuevos adaptadores.
- RFM será la base mínima de features para el primer modelo.
- La segmentación se implementará mediante Machine Learning en Python.
- Azure Machine Learning administrará y desplegará el modelo una vez validado localmente.
- Azure OpenAI se utilizará para explicación y recomendaciones, no para decidir los clusters.
- React será el frontend y consumirá únicamente la API principal del backend.
- FastAPI será la tecnología principal del backend y de la API simulada, manteniendo ambos componentes separados por responsabilidad.